Recall & Practice

Module 1: Python Foundations

```python
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))  # 6
```

By the End of Today, You'll Be Able To…

1

Retrieve the last four lessons, cold

2

Write a function with flexible arguments

You'll Know These by the End of Today

`*args`

Collects extra positional arguments into a tuple.

`**kwargs`

Collects extra named arguments into a dict.

FOUR LESSONS DEEP

Prove It’s Still Solid

Serialization, validation, error handling, and summarizing — retrieve them without a safety net.

Today: Recall, Then Two New Ideas

RECALL

Quick-fire recall on the last four lessons.

NEW

Flexible arguments, then prompting AI.

SUPPORT

Ask questions any time — no pressure.

CHECK FOR UNDERSTANDING

What does `json.load()` do?

A Reads a file and returns a Python value

B Converts a Python value to a JSON string

C Deletes a JSON file

ANSWER KEY

`json.load()` Recall

Correct: Reads a file and returns a real Python value

`json.load()` deserializes JSON from a file into a dict/list.
`json.dump()` is the reverse.

QUICK CHALLENGE

A line has 3 fields, but the third is `"x"`. Which check catches it?

A Field count

B `.isdigit()`

C Neither

ANSWER KEY

Which Check Catches It

Correct answer: `.isdigit()`

Three fields passes the count check. `"x".isdigit()` is `False`, so that check catches the bad value.

QUICK CHALLENGE

A conversion uses bare `except:` What's the safer fix?

A

```
except
ValueError:
```

B

Remove `try/except`

C

Add another bare `except`

ANSWER KEY

Fix the Bare `except`

Correct: Change it to `except ValueError:`

Naming the exact exception prevents unrelated bugs from being silently swallowed.

CHECK FOR UNDERSTANDING

After `chmod +x report.py`, typing `report.py` says `command not found`. Why?

A

It needs a `.sh` ending

B

Without `./`, the shell only searches `PATH`

C

`chmod +x` wore off; run it again

ANSWER KEY

Shebang Recall: Why `./`

Correct answer: B. No `./`, so only `PATH` is searched

Bare names are only searched for in `PATH` folders. `./` points at this folder, so `report.py` is found.

TIP

“Why Validate the Data Before You Summarize It?”

VALIDATE

```
validate_batch(lines)
```

SUMMARIZE

```
summarize_batch(games)
```

Answer with QAR: your first question, your assumption, the risk.

MODEL ANSWER

Model Answer: QAR

QUESTION

Where does this file come from, and has anyone checked it?

ASSUMPTION

It's raw input, so lines like `ADD,Checkers,two` slip in.

RISK

Summarize unchecked rows and the report is built on bad data.

Now, Something New

Every function so far had a fixed parameter list. What if you don’t know how many arguments you’ll get?

`*args` Handles an Unknown Number of Positional Arguments

FIXED

A normal function decides the exact argument count up front.

`*args` Handles an Unknown Number of Positional Arguments

FIXED

A normal function decides the exact argument count up front.

`*args`

Like a recipe that scales: work with however many servings arrive.

`**kwargs` Collects Extra Arguments by Name

TWO ASTERISKS
Collect extras by name.

`**kwargs` Collects Extra Arguments by Name

TWO ASTERISKS
Collect extras by name.

INSIDE

They land as a dict.

`**kwargs` Collects Extra Arguments by Name

TWO ASTERISKS
Collect extras by name.

INSIDE
They land as a dict.

WHY
Same flexibility, now with labels.

CODE ALONG

Open the Code-Along

Open this file:
`Module1_Lesson12_recall-practice-functions-validation-try-except-serialization_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Any Number of Arguments With `*args`
   One asterisk collects any number of extra positional arguments into a tuple.
2. Any Number of Named Arguments With `**kwargs`
   Two asterisks collect any number of named arguments into a dict.

Any Number of Arguments With `*args`

python

```python
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))
print(total(10, 20))
print(total())
```

Any Number of Named Arguments With **kwargs

python

```python
def describe_game(title, **details):
    print(title)
    for key, value in details.items():
        print(f"  {key}: {value}")

describe_game("Chess", players=2,
category="Strategy")
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

QUICK CHALLENGE

With the total() code, what does `total(4, 4, 4, 4)` return?

A 4

B 16

C An error

ANSWER KEY

`*args` Trace

Correct answer: 16

`*numbers` collects the four values into the tuple `(4, 4, 4, 4)`, and `sum()` adds them.

Watch It Run: `*numbers` Packs a Tuple

Step 1 of 6

We call total with three arguments: 1, 2, 3

```
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3))
print(total(10, 20))
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

QUICK CHALLENGE

With the describe_game() code, what does `describe_game("Uno", players=4)` print?

A

An error

B

“Uno” only

C

“Uno” then
“players: 4”

ANSWER KEY

`**kwargs` Trace

Correct: “Uno” then “players: 4”

`**details` contains only the named arguments actually passed. `category` is optional here, so no error occurs.

`*args` vs. `**kwargs`

`*args`

Extras by position become a tuple

`**kwargs`

Extras by name become a dict

IN COMMON: both make a function flexible when the argument count can vary.

TIP

“When Would You Use `*args` Over a List?”

```
def total(*numbers)
```

Call with separate values when the count varies.

```
def total(numbers_list)
```

Pass one collection when the list already exists.

Answer with QAR: your first question, your assumption, the risk.

MODEL ANSWER

Model Answer: QAR

QUESTION

Do the values arrive typed into the call, or in a list?

ASSUMPTION

Typed in one by one, and the count changes per call.

RISK

If a list arrives, `total([1, 2])` raises a TypeError.

Key Terms — Recap

`*args`

Extra positional arguments, collected as a tuple

`**kwargs`

Extra named arguments, collected as a dict

Prompting: Tags, Plans and Tests

Ask AI to plan and to test, not to write your code. Then check what it tells you.

CHECK FOR UNDERSTANDING

Your code and full error go in which TCEO part?

A

Evidence

B

Context

C

Output

ANSWER KEY

Prompting Recall: TCEO

Correct answer: Evidence

Evidence is the real thing, like your code, the full error, or a few rows.
One example of the answer shape goes in Output.

XML TAGS

Wrap Each Part of a Prompt in a Tag

THE PROMPT

```
<task>Find the row that breaks my sum.</task>
<context>I add hours with float().</context>
<data>E901,Test A,8
E902,Test B,</data>
<example>row 7: text in hours</example>
<output>One line per bad row. No code.</output>
```

WHY TAGS

Each part stays separate. Data rows never read as instructions.

Close each tag with a slash, like `</data>`.

Plan Mode: Plan, Approve, Then Code

1. PLAN

Ask for the steps in order. Say: `Plan only, no code.`

Plan Mode: Plan, Approve, Then Code

1. PLAN

Ask for the steps in order. Say: `Plan only, no code.`

2. APPROVE

Read each step. Ask about any step you don't get. Fix it, then approve.

Plan Mode: Plan, Approve, Then Code

1. PLAN

Ask for the steps in order. Say: `Plan only, no code.`

2. APPROVE

Read each step. Ask about any step you don't get. Fix it, then approve.

3. YOU CODE

Write it yourself, one step at a time. Run it after each step.

Module 1 rule: AI can plan, explain and test. You write the code.

Ask for Tests. You Decide the Answers.

ASK FOR

```
Do not write code. Give me 5
test inputs, some right at the
boundary. Do not say what it
returns.
```

YOU DECIDE

Work out each answer on paper.
Run your function on every input.
Keep any test that surprised you.

AI is good at listing tricky inputs. It is not trusted to grade them.

DATA PRIVACY

Never Paste Real Personal Data Into AI

NEVER PASTE

Real names, emails, phone numbers, IDs
Passwords and API keys
Anything marked private

PASTE MADE-UP ROWS

```
E901,Test Person A,8
E902,Test Person B,
```

Same shape. Invented names and IDs.

Before you send, reread it for real names, emails, IDs and keys.

TURN AND TALK

Hands-On: Ask AI for Edge-Case Tests

```python
def validate_batch(lines):
    for line_number, line in enumerate(lines, 1):
        fields = line.split(",")
        if len(fields) != 3:
            print(f"Line {line_number}: bad format")
        elif not fields[2].isdigit():
            print(f"Line {line_number}: bad players")
        else:
            action, title, players = fields
            print(f"Line {line_number}: OK, {title}")
```

IN PAIRS: 6 MINUTES

1. Write a prompt with `<task>`, `<rules>`, `<function>` and `<output>` tags. Ask for 6 tricky test lines. No code.
2. Use the rules to decide each line's verdict before you run anything.
3. Run two lines through `validate_batch`. Did it agree with you?

The rules: 3 fields, and the third one is all digits.

MODEL ANSWER

Model Answer: A Tagged Prompt, Two Lines Run

THE PROMPT

```
<task>List tricky test lines for my validator.
Do not write code.</task>
<rules>3 fields. The third is all digits.</rules>
<function>[paste validate_batch]</function>
<output>6 lines, one reason each. Do not say
what my function returns.</output>
```

TWO LINES WE RAN

`ADD,,4` printed OK. The rules never said a title can't be empty.

`ADD,Chess, 4` printed bad players. The space breaks `isdigit()`.

Your rules decide. The run shows what the code does.

Your Prompt Checklist, Part 2

TAGS

Wrap task, data and output in their own tags.

PLAN FIRST

Approve a plan. Then you write the code.

TEST SAFELY

Ask for edge cases. Paste made-up rows only.

So Far, We've Covered…

RETRIEVED

Four lessons, confirmed solid.

APPLIED

All of it powers the project's functions.

NEXT

Build a class with a partner.